CO1035: Game Concepts
Lab 2A – Texture Manipulation – Mouse Control – Custom Camera Control

1. Introduction

- In the previous lab activities, we introduced the TL-Engine and used it to manipulate some simple 3D graphics. We have also used an FPS camera, which is controlled using the cursor keys and the mouse.
- It would be better if you could set up the camera controls, speed, and rotation based on your own preferences and the machine you are working on. Therefore, in this session you will create your own camera controls.
- But first, you will learn to change the textures/skin on a model in the Tl-Engine.
- Remember you will need to keep the code that you write. For example, you will be expected to reuse the camera control code you write today for use in later weeks. Therefore, as always, make sure you save copies of your code and projects in secure, accessible locations.

---

2. Texture Manipulation

a. Create a new project named **Lab2A_TextureManipulation_Project**. Open the project.

b. It is possible to change the skin on a model (the image that is drawn on the geometry). To do so, you will need to use this method on the model:

```
void SetSkin( const string& sFileName );
//    Example usage: sphere->SetSkin( "Marine2.jpg" );
//    only works for models that use a single texture.
```

c. The TL-Engine sets a default location for all of its media files. This location is set in the initialization section as part of the template:

```
myEngine->AddMediaFolder( "C\\..." );
```

The path provided as a parameter to the function above is the location of the media files. This will probably be a folder in C on your home computer. If you look inside the directory, you can examine the media files installed with TL-Engine.

d. Now add and display a sphere in your scene so that you can change its skin. Here are some examples of textures designed to be used with the sphere: **Clouds.jpg; EarthHi.jpg, EarthNight.jpg, EarthPlain.jpg, Jupiter.jpg, Mars.jpg, Saturn.jpg, Pluto.jpg**.

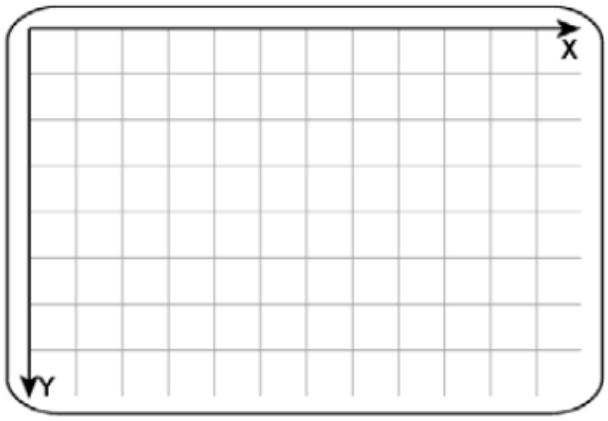
e. Save your project.

---

3. Mouse Control

a. Previously, we used 'if' statements to check whether mouse buttons are held or pressed.

b. The 3D engine can also provide the position and movement of the mouse cursor using the X and Y axes of the screen, which are different than the axes in the 3D engine. The screen axes look like the image below:



c. To get mouse information from the engine, use these methods:

```
// Returns the X position of the mouse (horizontal position relative to
// the top-left of the 3D engine window).
int GetMouseX();
// Returns the Y position of the mouse (vertical position relative to
// the top-left of the 3D engine window).
int GetMouseY();
// Returns the X mouse movement since the last call to this function
int GetMouseMovementX();
// Returns the Y mouse movement since the last call to this function
int GetMouseMovementY();
```

The first two methods return the X and Y position of the mouse cursor in the engine window. This can be used for clicking on buttons or icons - we won't be using these methods right now. The second two methods tell you how much the mouse has been moved in the X and Y

directions (since the last time you asked). This is a common way to control the rotation of 3D objects.

d. Open the lab project, **Lab1C_Controls_Project**, so that we can add code to handle mouse input for control.

e. We want to use the mouse input in **Lab1C_Controls_Project**, but the FPS camera is using the mouse, so change the camera type from '**kFPS'** to '**kManual'**. Remember from the previous lab that this means you can't move the camera anymore, so remove the "**Grid.x**" mesh again so you can see better.

f. Remove the lines of code that rotate the arrow in your program, including the lines to test if the keys 'Z' and 'X' are pressed. Replace them with two lines similar to these (but use your own model name):

```
int mouseMoveX = myEngine->GetMouseMovementX();
arrow->RotateY(mouseMoveX * 0.1); // the 0.1 reduces the rotation speed
```

This code should not be inside an 'if' statement – it should always be executed.

g. Run the program; you will be able to control the cube rotation with the mouse – but only when the mouse cursor is in the engine window; we will fix this later.

h. Notice that we use the mouse X-axis movement to control rotation around the 3D engine Y-axis – this is confusing, but correct: Try using the mouse Y movement instead to see what happens (move the mouse up and down). It is often best to experiment in these cases.

i. Now add a line of code to control the arrow's X rotation with the mouse Y movement.

j. We now have complete control over the arrow using the mouse and keys. Save your project.

---

4. Custom Camera Control

a. Create a new TL_Engine project named **Lab2A_CameraControl_Project**

b. Create and display models of your own choice. Set the camera type to 'kManual' again, although this camera is fixed by default you can add code to position, move and rotate a it using exactly the same methods as you have used with models.

c. First, set up a value to control the speed of the camera. Declare a constant called "kCameraSpeed" outside the main function just under the statementusing namespace

tle; (constants are declared outside the main function). Initialize "kCameraSpeed" to a suitable value using the const keyword, for example:

```
const float kCameraSpeed = 0.001f;
```

This specifies that the variable's value is constant and tells the compiler to prevent the programmer from modifying it

d. Whenever the compiler sees the word "kCameraSpeed" it will use the numerical value that you specified.

e. Now set up another constant called "kCameraRotation" and initialize it to a suitable value.

f. Control the camera's movement with the same keys that you used for the cube: 'A' & 'D' = left and right, 'W' and 'S' = forward and backward, 'Q' & 'E' = up and down. You should use the local movement methods.

g. Next use the mouse movement to control the camera rotation.

h. Your final program should be very similar to the previous one.

i. You should have recreated a simple FPS camera, but we still have the problem that it only works when the mouse cursor is in the engine window.

j. Add this line in the initialization section within the 'int main' function:

```
myEngine->StartMouseCapture();
```

This will switch off the mouse cursor and send all mouse information to the engine. You can restore the mouse with:

```
myEngine->StopMouseCapture();
```

k. Create a toggle button to switch between using a captured mouse (cursor invisible) and using a non-captured mouse (cursor is visible). I would suggest using the TAB key. **Hint:** You will need to use an 'if' statement to check for the appropriate key press inside the game loop.

l. You will need to reuse the camera setup in later programs you write. The two constants which you have declared mean that the code is flexible. You can use it in new projects and merely adjust the constants to suit the speed of the computer you are working on.